

软件过程与管理课程作业

作业编号	Assgt_01	学 号	2023141461017
学生姓名	杨昊宇	专 业	软件工程
年 级	2023 级	学 院	软件学院
题 目	NovelCraft AI 小说创作平台软件过程定义		

一、项目概述

本项目为 NovelCraft AI 小说创作平台，是一个基于生成式 AI 的小说创作辅助平台，支持用户通过简单提示词自动生成完整世界观、角色设定、剧情内容，并提供角色对话扩展功能，实现从创意到内容的智能化创作闭环。

项目采用前后端分离架构，前端使用 React 18 + TypeScript + Vite + Tailwind CSS，后端使用 Spring Boot 3.2.0 + MongoDB + Redis + PostgreSQL，AI 能力通过统一服务层封装，支持 SSE 流式输出。

项目团队共 5 人，采用敏捷开发模式，历时约 12 周完成从需求分析到上线交付的全流程。

二、软件过程定义

根据 ISO/IEC 12207:2008《系统和软件工程——软件生命周期过程》标准，结合项目实际情况，我对 NovelCraft 项目的软件过程进行如下定义和剪裁。

2.1 过程定义来源

本项目的软件过程定义主要来源于以下三个国际标准：

- ISO/IEC 12207:2008《系统和软件工程——软件生命周期过程》：提供了软件生命周期过程的通用框架，包含主要过程、支持过程和组织过程三大类，共 17 个过程。
- ISO/IEC 15504 (SPICE)《信息技术——过程评估》：提供了软件过程评估的参考模型，用于评估和改进软件开发过程的能力成熟度。
- ISO/IEC 33001:2015《信息技术——过程评估——概念和术语》：作为 ISO/IEC 15504 的演进版本，提供了过程评估的概念框架和术语体系，支持对过程质量特性的评估。

在定义过程中，我以 ISO/IEC 12207 的生命周期过程框架为基础，结合 ISO/IEC 33001 的过程评估理念，对标准过程进行剪裁，形成适合本项目特点的软件过程定义。

2.2 采用的主要过程（Primary Processes）

根据 ISO/IEC 12207，主要过程包括获取、供应、开发、运行和维护五个过程。本项目作为学生课程实践项目，主要涉及以下三个主要过程：

- 【开发过程（Development Process）】——核心过程：定义开发者定义和开发软件产品的活动。在本项目中，开发过程是核心过程，涵盖从需求分析、系统设计、编码实现到测试验证的全流程。
- 【供应过程（Supply Process）】——简化采用：定义供方向需方提供系统、软件产品或软件服务的活动。在本项目中，由于项目为学生实践项目，不存在正式的采购和供应关系，因此将供应过程简化为项目计划书的编写和团队内部的需求确认。
- 【维护过程（Maintenance Process）】——预留接口：定义维护者对软件产品进行维护服务的活动。在本项目中，维护过程主要体现为上线后的 72 小时监控和后续的问题修复，但由于项目周期限制，未开展完整的维护活动。

【剪裁说明】：获取过程和运行过程在本项目中未采用。获取过程因项目为内部课程实践，不存在外部采购需求；运行过程因项目为 Web 应用，部署在云服务器上，未涉及传统意义上的"运行者"角色。

2.3 采用的支持过程（Support Processes）

根据 ISO/IEC 12207，支持过程包括文档编制、配置管理、质量保证、验证、确认、联合评审、审计和问题解决等八个过程。本项目采用了以下支持过程：

- 【文档编制过程 (Documentation Process)】——完整采用：对生存周期过程所产生的信息进行记录。本项目编写了项目计划书、系统设计文档、API 接口文档、测试用例与报告、部署与运维手册等。
- 【配置管理过程 (Configuration Management Process)】——完整采用：定义配置管理活动。本项目使用 Git 进行版本控制，采用 Git Flow 规范，主分支 (main) 始终保持可发布状态，开发分支 (develop) 用于集成日常开发。
- 【质量保证过程 (Quality Assurance Process)】——部分采用：定义保证软件产品和过程符合规定要求的活动。本项目通过代码审查、静态检查、单元测试覆盖率要求 ($\geq 80\%$) 等方式保证质量，但未设立独立的质量保证团队。
- 【验证过程 (Verification Process)】——完整采用：定义对软件产品进行验证的活动。本项目通过接口测试、联调测试、配置读取测试等方式进行验证。
- 【确认过程 (Validation Process)】——完整采用：定义对软件产品进行确认的活动。本项目通过功能演示、用户试用反馈等方式进行确认。
- 【联合评审过程 (Joint Review Process)】——部分采用：定义对某项活动的状态和产品进行评价的活动。本项目通过每周项目例会和里程碑评审进行联合评审。
- 【问题解决过程 (Problem Resolution Process)】——完整采用：定义对开发过程中发现的问题进行分析与排除的过程。本项目建立了问题跟踪机制，通过钉钉群和 GitHub Issues 进行问题记录和闭环。

【剪裁说明】：审计过程在本项目中未采用。因项目为学生课程实践，不存在独立的审计方，审计活动由项目组长兼任完成，未单独设立审计过程。

2.4 采用的组织过程 (Organizational Processes)

根据 ISO/IEC 12207，组织过程包括管理、基础设施、改进和培训四个过程。
本项目采用了以下组织过程：

- **【管理过程 (Management Process)】**——完整采用：定义在生存周期过程中管理的基本活动。本项目采用敏捷开发模式，每周三召开项目例会，使用 Jira/钉钉进行任务跟踪和进度管理。
- **【基础设施过程 (Infrastructure Process)】**——一部分采用：定义建立生存周期过程的基础结构所需的基本活动。本项目建立了共享文档库 (Notion/Google Drive)，配置了开发环境 (Git、Docker、IDEA、Postman)。
- **【改进过程 (Improvement Process)】**——一部分采用：定义为了建立、测量、控制和改进其生存周期过程需完成的基本活动。本项目通过每周复盘和项目结项复盘会进行过程改进。
- **【培训过程 (Training Process)】**——简化采用：定义为提供适当的人员培训所需的活动。本项目通过技术分享和结对编程进行内部培训，未编写正式的培训资料。

2.5 过程定义总览表

过程类别	过程名称	采用方式	说明
主要过程	开发过程	完整采用	核心过程，涵盖需求分析到测试验证
主要过程	供应过程	简化采用	简化为项目计划书编写和需求确认
主要过程	维护过程	预留接口	上线后 72 小时监控和问题修复
主要过程	获取过程	未采用	项目为内部实践，无外部采购
主要过程	运行过程	未采用	Web 应用部署在云服务器，无传统运行者
支持过程	文档编制	完整采用	项目计划书、设计文档、API 文档等
支持过程	配置管理	完整采用	Git + Git Flow 版

			本控制
支持过程	质量保证	部分采用	代码审查、静态检查、单元测试
支持过程	验证过程	完整采用	接口测试、联调测试
支持过程	确认过程	完整采用	功能演示、用户试用反馈
支持过程	联合评审	部分采用	每周例会和里程碑评审
支持过程	问题解决	完整采用	钉钉群 + GitHub Issues 跟踪
支持过程	审计过程	未采用	无独立审计方，由组长兼任

三、剪裁的想法与理由

在对 ISO/IEC 12207 标准过程进行剪裁时，我遵循了以下原则和思路：

3.1 剪裁原则

- **【适用性原则】**：剪裁应基于项目的实际特点和需求，而非简单照搬标准。
本项目为学生课程实践项目，团队规模小（5 人）、周期短（12 周）、资源有限，因此需要适当简化部分过程。
- **【完整性原则】**：剪裁后的过程应能覆盖项目全生命周期，确保核心活动不缺失。即使简化某些过程，也要保证项目能够顺利推进并达到预期目标。
- **【经济性原则】**：剪裁应考虑过程执行的成本和收益，避免过度过程化导致效率低下。对于小型项目，过于繁琐的过程反而会成为负担。
- **【可追溯原则】**：剪裁决策应有明确的理由和记录，便于后续评估和改进。

3.2 具体剪裁想法

(1) 主要过程的剪裁

开发过程作为核心过程完整保留，因为无论项目规模大小，软件开发的基本活动（需求、设计、编码、测试）都是不可或缺的。供应过程简化为项目计划书编写，因为学生项目不存在正式的甲乙双方关系，但需求确认和计划制定仍然必要。维护过程仅保留上线后监控，因为课程项目周期有限，长期维护不在项目范围内。获取过程和运行过程直接剪裁，因为项目不涉及外部采购和传统运行角色。

(2) 支持过程的剪裁

文档编制、配置管理、验证、确认、问题解决等过程完整保留，因为这些是保障项目质量的基础过程，无论项目规模大小都应执行。质量保证和联合评审部分采用，因为学生项目没有独立的质量保证团队和正式的评审会议，但可以通过代码审

查和例会形式实现类似功能。审计过程完全剪裁，因为不存在独立的审计方，且项目规模小，审计的必要性不高。

(3) 组织过程的剪裁

管理过程完整保留，因为项目管理是确保项目按时交付的关键。基础设施过程部分采用，因为学生项目可以借助现有的开发工具和环境，不需要从零搭建完整的基础设施。改进过程部分采用，因为项目周期短，难以开展系统性的过程改进，但可以通过复盘进行经验沉淀。培训过程简化采用，因为团队成员已具备基本技术能力，培训主要通过技术分享和结对编程完成。

3.3 剪裁依据

本次剪裁主要依据以下因素：

- **【项目规模】**：5 人团队，12 周周期，属于小型项目，不适合采用大型企业级项目的完整过程。
- **【项目性质】**：学生课程实践项目，以学习和锻炼为主要目的，过程定义应兼顾教学要求和实际可行性。
- **【技术特点】**：AI 类项目，技术不确定性高，需要预留足够的调试和联调时间，因此部分过程需要简化。
- **【团队能力】**：团队成员技术能力存在差异，过程定义需要考虑团队的学习曲线和协作效率。
- **【资源约束】**：预算有限（API 调用费用月度上限 500 元），需要合理分配资源，避免过程执行成本过高。

3.4 与 ISO/IEC 33001 的关联

ISO/IEC 33001:2015 提供了过程评估的概念框架，强调基于客观证据对过程质量特性进行评估。在本项目的软件过程定义中，我借鉴了 ISO/IEC 33001 的以下理念：

- **【过程质量特性】**：在定义过程时，不仅关注过程是否被执行，还关注过程的执行效果（如单元测试覆盖率 $\geq 80\%$ 、接口可用性 $\geq 99\%$ 等）。
- **【能力等级】**：虽然本项目未进行正式的过程能力评估，但在过程定义中预留了度量指标（如代码审查覆盖率、缺陷修复时效等），为后续评估提供数据基础。
- **【持续改进】**：通过每周复盘和项目结项复盘，对过程执行情况进行回顾和改进，体现了 ISO/IEC 33001 倡导的持续改进理念。

3.5 过程定义的层次性

根据课程所学，软件过程定义具有层次性。本项目的过程定义体现了以下层次：

- **【组织级过程】**：项目遵循团队约定的 Git Flow 规范、代码审查制度、例会制度等组织级过程。
- **【项目级过程】**：针对 NovelCraft 项目的特点，定义了具体的需求分析、设计、开发、测试、部署等项目级过程。
- **【任务级过程】**：将项目级过程进一步细化为可执行的任务，如"开发小说推荐算法接口"包含需求理解、方案设计、编码实现、单元测试、接口联调等具体任务。

这种层次化的过程定义使得过程既具有规范性，又具有灵活性，能够适应不同粒度的管理需求。

四、总结

通过对 ISO/IEC 12207、ISO/IEC 15504 和 ISO/IEC 33001 三个标准的学习和理解，我结合 NovelCraft AI 小说创作平台项目的实际情况，对软件生命周期过程进行了定义和剪裁。

在主要过程中，完整采用了开发过程，简化了供应过程和维护过程，剪裁了获取过程和运行过程。在支持过程中，完整采用了文档编制、配置管理、验证、确认、问题解决等过程，部分采用了质量保证和联合评审过程，剪裁了审计过程。在组织过程中，完整采用了管理过程，部分采用了基础设施和改进过程，简化了培训过程。

剪裁的核心思路是：在保证项目核心活动完整执行的前提下，根据项目规模、性质、技术特点、团队能力和资源约束等因素，对标准过程进行适当简化和调整。剪裁不是对标准的否定，而是对标准的灵活应用，目的是使过程定义更加贴合项目实际，提高过程执行的有效性和经济性。

通过本次作业，我深刻理解了软件过程定义的重要性和复杂性。一个好的软件过程定义，既要遵循国际标准的框架和指导，又要结合项目实际进行合理剪裁，在规范性和灵活性之间找到平衡。同时，过程定义不是一成不变的，应根据项目进展和反馈进行持续改进，这也是 ISO/IEC 33001 所倡导的持续改进理念的体现。